

Project Management Plan

Project Title: MataBank Banking Website

Team Members: Lernik Avetisyan, Gus Axelson, Lance Jimenez, Anthony Taylor

1. Project Objectives

- **Automatic and Manual Wallet Setup**
 - Backend generates a unique card number, expiration date, and CVV when users create a new wallet.
 - Users can add their own existing cards manually.
- **Easy Money Management**
 - View current balance and full transaction history.
 - Deposit and withdraw funds with real-time updates.
- **Strong Security & Access Control**
 - Require users to log in before accessing wallets.
 - Enforce role-based access so only the account owner can manage each wallet.
- **User-Friendly Interface**
 - Clean, responsive design that works on desktops.
 - Immediate input checks (card number format, date, CVV) with clear error messages.
- **Reliable Performance**
 - Fast page loads (< 200 ms) and 99.9 % uptime.
 - Prevent duplicate cards or balance errors via backend checks.
- **Transaction Recording**
 - Record every deposit and withdrawal as a Transaction entry, so users, employees and admins can audit all activity.

2. Project Scope

During the project development process, there will be a focus on key features and deliverables.

In Scope:

- Account Registration, Login, and Management
- Creation of Multiple Wallets
- Incoming Transfers Added to Wallet
- Outgoing Transfers Subtracted from Wallet
- Display Analysis of Spending Trends
- Role-Based Access Control

Out of Scope:

- Dedicated Matabank Mobile App (strictly a Website Platform)

3. Work Breakdown Structure (WBS)

Phase	Tasks
1. Requirements Gathering	Conduct stakeholder interviews, determine functional and non-functional requirements for a System Requirements Specification (SRS)
2. Design	Begin UI/UX planning, system architecture diagrams, user flow diagrams,
3. Development	Frontend and backend development, implement features like deposit, withdraw, money transfers, and security features
4. Testing	Perform unit, integration, and system testing, verify requirements, confirm security measures
5. Deployment	Deploy onto a secure web hosting service, and monitor launch for possible issues
6. Maintenance	Track website performance, monitor for bugs, frequently update security

4. Project Schedule

Week	Activities
1-3	Requirement Planning, stakeholder meetings, SRS creation
4-5	System design, wireframe models, database setup
5-9	Implementation of functional and non-functional requirements
10-11	Testing of the system and User Acceptance testing before deployment
12	Final deployment
13	Final project presentation, completed documentation

5. Roles and Responsibilities

To facilitate organization, there will be key roles and responsibilities assigned to all members involved in the project development process.

Role	Responsibilities
Project Manager	Contributes to project advancement by planning project activities, overseeing progress, and facilitating communication between all members.
Frontend Developer	Develops web user interfaces of the MataBank Banking Website using the applicable tools: CSS, Javascript, and HTML
Backend Developer	Develops logic of functions involved in money transfers, account management, and spending trend data
Test Engineer	Writes and executes test cases to ensure project quality while logging bugs discovered during testing.

6. Risk Management Plan

Risk	Likelihood	Impact	Mitigation Strategy
Duplicate card number collision	Medium	High	Wrap “check + create” in a database transaction with a lock, retry up to 10 times, and alert on collisions.
Security vulnerabilities	Low	Critical	Run automated security scans, enforce HTTPS, sanitize inputs, conduct quarterly pen tests.
Server downtime or slow performance	Medium	Medium	Deploy on auto-scaling servers with health checks, use caching, and maintain daily backups.
Requirements creep (scope changes)	Medium	Medium	Freeze scope after SRS, review all change requests at sprint boundaries and adjust schedule only by agreement.
Team member turnover	Medium	Medium	Cross-train team members, keep up-to-date documents, and include a 10 % time buffer for onboarding.

Database failure or data corruption	Low	Critical	Enable automated daily backups, use a replicated database setup, and test restores regularly.
Browser compatibility issues	Low	Low	Test UI on major browsers and devices, use a CSS framework with builtin cross-browser support, and fix issues early.
Overlapping transaction errors	Medium	High	Use database transactions and row-level locks for deposit/withdraw operations, and add retry logic on conflicts.

7. Communication Plan

Activity	Audience	Frequency	Format
Daily Stand-up	Dev Team	Every Week	15 min Zoom or Call
Weekly Status Update	PM & Stakeholders	Every Week	Email summary + project dashboard link
Monthly Progress Report	PM & Stakeholders	Every month	PDF report + dashboard link
Show New Features	Dev Team, PM, Stakeholders	Every 2 weeks	Live demo + quick feedback session
Code Review	Dev Team	Every Week	GitHub pull-request reviews
Track & Solve Issues	Dev & QA Team	Ongoing	Slack channel + Jira tickets
Reflect & Improve	Dev Team & QA	End of each 2-week cycle	30-min meeting to discuss lessons learned
Design Review	Dev Team, Designer, PM	Every month	Video call with screen share
Security Check-in	Security Lead & PM	Every month	30-min call + short security report

Release Announcement	All Users & Stakeholders	With each release	Email blast + in-app notification
Urgent Alerts	Dev, QA, PM	If needed	Slack ping + short Zoom call
Final Presentation	All Stakeholders	End of project	Live demo + overall summary report

8. Tools and Technologies

Development:

- **VSCode:** code editor IDE
- **Web Forums:** for specific problem solving
- **MySQL:** Database
- **Javascript:** Front and Backends
- **HTML:** Front end
- **CSS:** Front End
- **Node.js:** Backend

Communication:

- **Discord:** main source of communication
- **Email:** extra communication (sharing docs)
- **Github:** sharing code between each other
- **Flash drive:** sharing code between each other

Testing:

- **Jest or Mocha:** used for unit testing JavaScript functions and modules
- **Supertest:** for testing API endpoints during integration testing
- **Cypress:** to automate full workflows like login, fund transfer, and viewing transaction history
- **OWASP ZAP (light mode):** to perform lightweight scans for common web vulnerabilities
- **Lighthouse (Chrome DevTools):** to evaluate site speed, accessibility, and performance
- **Uptime Robot:** to evaluate uptime.

9. Success Criteria

Unit and Integration Testing

- **Criterion:** At least 95% of unit and integration test cases pass.
- **Measurement:** Automated testing reports from Jest and SuperTest
- **Goal:** Ensure each component and its integrations behave as expected in isolation and in combination.

System and End-to-End Testing

- **Criterion:** At least 90% of system and E2E test cases pass, with 0 critical or high-priority defects open.
- **Measurement:** Test case execution results from Cypress.
- **Goal:** Validate the full functionality of the banking workflow (e.g., login, account management, transactions).

Security Testing

- **Criterion:** No critical or high-severity vulnerabilities detected in the final security audit.
- **Measurement:** Reports from penetration testing, using OWASP ZAP.
- **Goal:** Ensure the system is resistant to common threats like XSS, SQL injection, and CSRF.

Performance Testing

- **Criterion:** System handles at least 1000 concurrent users with response time < 2 seconds for 95% of requests
- **Measurement:** Load test reports using Lighthouse.
- **Goal:** Ensure site responsiveness and scalability under load.

Uptime and Availability

- **Criterion:** Website maintains 99.9% uptime during the first 30 days post-launch.
- **Measurement:** Monitoring logs from tools like UptimeRobot
- **Goal:** Validate operational stability of the deployed system.